VISIO TECH · INGÉNIERIE LOGICIELLE

LEYISA SCHOOL V1

Prêt à développer

Guide de lecture facile et cahier de travail

Notre fil conducteur : inscrire un élève, attribuer un enseignant, saisir et contrôler ses notes, puis produire un bulletin fiable.

Édition pédagogique et technique · 11 octobre 2026

Équipe : Sidonie · Franck · Jonathan · Kevin · Melvin · Joel

> *Cette version réécrit le Ready-to-Code du 7 octobre. Elle sert à comprendre et à préparer le développement. Elle ne prouve ni que le logiciel est terminé, ni que les règles officielles de l'école sont confirmées.*

Comment utiliser ce guide

Commencer par le parcours, puis consulter les détails selon sa mission.

Trois niveaux de lecture

1. Pour découvrir Leyisa. Lire « Notre objectif », « Les utilisateurs », « Le parcours » et « La démonstration ». Aucun bagage informatique n'est nécessaire.

2. Pour développer un écran. Lire la fiche US correspondante, puis les règles, les permissions et les données de ce domaine.

3. Pour relire ou tester. Prendre les quatre vérifications de chaque fiche US, y ajouter les refus d'accès et les cas de concurrence concernés.

Ce qui est décidé et ce qui ne l'est pas encore

Mention	Ce qu'elle signifie
Fait connu	Information présente dans les documents du projet.
Proposition V1	Règle proposée pour la démonstration. L'équipe doit l'adopter.
À confirmer avec l'école	Il manque la décision réelle de l'établissement.
À réaliser	Conception écrite, mais pas encore programmée ou testée.

> *Les codes US-xxx (fonctionnalité), Rxx (règle) et Cxx (contrat) sont conservés pour permettre aux développeurs de se retrouver dans le dossier original de 77 pages.*

1. Notre objectif

Une école, un parcours complet, une démonstration contrôlée.

Leyisa School vise un seul établissement. L'application rassemble les informations utiles : classes, élèves, enseignants, évaluations, notes, validations et bulletins.

Le parcours que nous voulons montrer

1. Un responsable prépare l'année, les périodes, les classes et les matières.
2. Le secrétariat crée les dossiers des élèves et les inscrit.
3. L'administrateur désigne les enseignants et leurs matières.
4. Chaque enseignant crée une évaluation et saisit les notes de sa classe.
5. La direction accepte les notes ou explique ce qui doit être corrigé.
6. La direction vérifie les résultats, clôture la classe pour la période et produit le bulletin.

Ce qui permettra de juger la démonstration

Trois élèves fictifs, une classe, deux matières, deux enseignants affectés, un rejet corrigé, une validation, une moyenne expliquée et un bulletin téléchargeable. Les informations doivent toujours exister après un redémarrage.

> *Tant que la formule et les règles réelles ne sont pas validées par l'école, le bulletin porte clairement « DÉMONSTRATION - NON OFFICIEL ».*

Ce que nous ne développons pas maintenant

Présences, paiements, espaces parents/élèves, discipline, bibliothèque, cantine, transport, statistiques avancées, gestion multi-école et correction officielle après clôture : ces fonctionnalités sont reportées.

2. Qui fait quoi dans l’école ?

Quatre rôles suffisent pour la démonstration.

Administrateur (ADM)

Crée les comptes et prépare l’année, les périodes, les classes, les matières et les affectations.

Limite importante. Ne peut pas approuver une note simplement parce qu’il administre l’application.

Secrétariat (SEC)

Recherche ou crée un élève, réalise l’inscription annuelle, consulte et télécharge un bulletin produit.

Limite importante. N’accède pas librement aux notes en cours.

Enseignant (ENS)

Voit ses propres classes et matières, crée les évaluations, enregistre puis soumet les notes.

Limite importante. Ne peut pas modifier les notes d’un collègue ni valider les siennes.

Direction (DIR)

Examine les notes envoyées, accepte ou rejette, contrôle les résultats et clôture la classe-période.

Limite importante. N’écrit pas directement une note à la place de l’enseignant.

> *Un même compte peut posséder plusieurs rôles. Il reste interdit d’approuver sa propre soumission, même avec les rôles ENS et DIR.*

Que signifie « clôturer » ?

Fermer les résultats d’une classe pour une période précise. La 6e A peut être clôturée pour la période 1 sans fermer la 6e B ni l’année scolaire entière.

3. Ce qu’il faut livrer en priorité

Le catalogue complet a 27 fonctions, mais le temps disponible est limité.

Le dossier du 7 octobre classe 27 fonctions comme indispensables à sa V1 complète. Notre échange le plus récent parle de cinq jours restants : il serait trompeur de promettre automatiquement ces 27 fonctions au même niveau de finition.

Priorité de la démonstration

- Connexion et déconnexion réelles avec quatre comptes de rôles différents.
- Année, classe, deux matières, élèves inscrits et enseignants affectés.
- Évaluation, saisie du brouillon, soumission et contrôle.
- Rejet motivé, correction, nouvelle soumission et validation.
- Calcul expliqué, clôture complète et bulletin figé.
- Contrôles d’accès, absence de doublons et historique des décisions.

Ce qui peut avoir une interface réduite

Recherche avancée, écrans complets d’administration, consultation détaillée de l’audit et options secondaires. Ils doivent rester dans le catalogue, mais l’équipe décide explicitement ce qu’elle reporte à la V1 complète.

> *Décision à prendre en réunion : quels écrans sont réellement exigés dans les cinq jours ? Reporter un écran ne permet pas de supprimer les contrôles de sécurité du parcours livré.*

4. Comment les besoins deviennent des modules

Un module est une partie du système qui a une responsabilité claire.

Nous avons d'abord étudié les actions des utilisateurs. Nous les avons regroupées par proximité : connexion avec accès ; école avec inscriptions ; évaluations avec notes ; validation avec résultats.

Parcours visible	Module informatique	Pourquoi cette séparation ?
Se connecter	M1 - Accès	Contrôler l'identité et les droits.
Préparer l'école / inscrire	M2 - Scolarité	Partager les années, classes et inscriptions.
Confier une matière	M3 - Enseignement	Savoir qui enseigne quoi.
Évaluer / saisir	M4 - Notes	Conserver les notes et leurs tentatives.
Contrôler / clôturer	M5 - Résultats	Décider et figer les résultats.
Produire un bulletin	M6 - Bulletins	Créer un PDF sans recalculer les notes.
Tracer les décisions	T1 - Audit	Garder une preuve des actions importantes.

Un seul backend, plusieurs modules

Les six modules restent dans une seule application. Ils ont des frontières claires : chacun possède ses données et communique avec les autres par un contrat, plutôt que de modifier leurs tables sans autorisation.

Technologies proposées, à confirmer au premier jour

React + Vite + TypeScript pour les écrans ; NestJS + TypeScript pour les traitements ; Prisma pour accéder à PostgreSQL ; Docker Compose pour démarrer les services. Ce sont les choix du dossier, pas des outils déjà installés et vérifiés chez les six développeurs.

> *Pas besoin de microservices, RabbitMQ, Redis ou Kubernetes pour démontrer ce parcours.*

5. Les données indispensables

Comprendre les mots avant de créer les tables.

Student - Élève

Dossier d'une personne : matricule et identité.

À retenir. Il ne contient pas de classe permanente.

SchoolYear - Année scolaire

Année utilisée pour organiser la scolarité.

À retenir. Une même classe peut exister dans deux années différentes.

SchoolClass - Classe annuelle

Groupe d'élèves dans une année donnée.

À retenir. La 6e A de cette année n'est pas celle de l'an prochain.

Enrollment - Inscription

Lien entre un élève, une classe et une année.

À retenir. Un élève ne peut pas être inscrit deux fois pour la même année dans la démo.

Subject / ClassSubject

Matière générale / matière prévue dans le programme d'une classe.

À retenir. Le programme explique quelles matières sont attendues.

Teacher / TeacherAssignment

Dossier enseignant / affectation d'une matière de classe.

À retenir. L'affectation donne le périmètre de travail de l'enseignant.

Evaluation / Grade

Épreuve et barème / note d'un élève inscrit.

À retenir. Une note doit correspondre à un élève réellement inscrit dans la classe.

Submission - Envoi

Copie des notes soumises à la direction, avec numéro de tentative.

À retenir. Un rejet n'efface pas la tentative précédente.

ClassPeriod

État de clôture pour une classe et une période.

À retenir. OPEN : on peut travailler ; CLOSED : les résultats sont figés.

ResultSnapshot / ReportCard

Photo définitive du résultat / bulletin qui présente cette photo.

À retenir. Le bulletin ne lit pas les notes courantes.

Pourquoi deux fichiers pour un même élève ?

Exemple : une élève était en 5e A en 2025-2026 et passe en 6e A en 2026-2027. Son dossier Student reste le même. Elle possède deux inscriptions Enrollment, chacune liée à sa vraie année.

> *Le diagramme UML corrigé V4 complète cette explication avec les cardinalités et les relations. Ce guide pédagogique ne le remplace pas par un dessin simplifié qui risquerait de déformer l'UML.*

6. Les 27 fonctions à construire

Chaque fiche garde son identifiant et quatre vérifications, exprimées en français courant.

Une « User Story » décrit une action utile à une personne. Les quatre points servent ensuite de petits tests. Le catalogue vient du dossier original ; le niveau de livraison en six jours reste à arbitrer.

US-001 · Se connecter

Tous les rôles · Responsable : Sidonie · PARCOURS

Objectif. Entrer dans son espace personnel.

Comment vérifier que cela fonctionne ?

1. Un compte autorisé ouvre une session.
2. Un mauvais mot de passe ou un compte désactivé ne donne aucun accès.
3. Sans connexion, une page privée refuse l’accès.
4. Les tentatives sont limitées et le mot de passe n’est jamais renvoyé.

US-002 · Se déconnecter

Tous les rôles · Responsable : Sidonie · PARCOURS

Objectif. Fermer son accès sur l’appareil.

Comment vérifier que cela fonctionne ?

1. La session est réellement arrêtée.
2. Une ancienne session ne permet plus de revenir.
3. Se déconnecter une deuxième fois ne bloque pas la page.
4. Les informations privées disparaissent de l’écran.

US-003 · Gérer les comptes

Administrateur · Responsable : Sidonie · PARCOURS

Objectif. Créer, désactiver et attribuer les rôles.

Comment vérifier que cela fonctionne ?

1. Créer un compte avec un identifiant unique et des rôles connus.
2. Refuser un identifiant déjà utilisé ou un rôle inconnu.
3. Désactiver ou changer un rôle invalide les anciennes sessions et laisse une trace.
4. Un non-administrateur est refusé ; le dernier ADM actif est protégé.

US-004 · Créer une année scolaire

Administrateur · Responsable : Jonathan · PARCOURS

Objectif. Préparer les années utilisées par Leyisa.

Comment vérifier que cela fonctionne ?

1. Un code unique et des dates correctes créent l'année.
2. Une date de fin avant la date de début est refusée.
3. Un code déjà existant est refusé.
4. L'année reste enregistrée et un non-ADM ne peut pas la créer.

US-005 · Définir les périodes

Administrateur · Responsable : Jonathan · PARCOURS

Objectif. Découper une année scolaire en périodes.

Comment vérifier que cela fonctionne ?

1. Une période indique son année, son ordre et ses dates.
2. Elle ne sort pas de l'année et ne chevauche pas une autre période.
3. Le même code ou ordre dans une année est refusé.
4. Les dates d'une période déjà utilisée ne se changent plus librement.

US-006 · Créer une classe annuelle

Administrateur · Responsable : Jonathan · PARCOURS

Objectif. Ouvrir une classe pour une année.

Comment vérifier que cela fonctionne ?

1. Le code est unique dans l'année.
2. Le même code est possible dans une autre année.
3. Une année inexistante est refusée.
4. Une classe utilisée ne se supprime pas et renommer ne déplace pas ses élèves.

US-007 · Créer une matière

Administrateur · Responsable : Jonathan · PARCOURS

Objectif. Tenir une liste commune des matières.

Comment vérifier que cela fonctionne ?

1. Le code et le nom sont valides.
2. Un code déjà présent est refusé.
3. La matière apparaît pour composer le programme d'une classe.
4. Un non-ADM ne la crée pas et une matière utilisée ne se supprime pas.

US-008 · Préparer le programme d’une classe

Administrateur · Responsable : Jonathan · PARCOURS

Objectif. Dire quelles matières doivent être évaluées.

Comment vérifier que cela fonctionne ?

1. Une matière peut être rattachée à une classe.
2. Le même lien deux fois est refusé.
3. Après la première évaluation de la classe, le programme est protégé.
4. Le programme est visible pour les affectations et les résultats.

US-009 · Rechercher un élève

Secrétariat · Responsable : Franck · PARCOURS

Objectif. Retrouver un dossier avant d’en créer un autre.

Comment vérifier que cela fonctionne ?

1. Chercher par nom ou matricule dans une liste organisée.
2. Si personne ne correspond, proposer de créer un dossier.
3. Des élèves ayant le même nom restent distincts.
4. Un enseignant ne peut pas parcourir tous les dossiers.

US-010 · Créer le dossier d’un élève

Secrétariat · Responsable : Franck · PARCOURS

Objectif. Enregistrer un élève avec son identité.

Comment vérifier que cela fonctionne ?

1. Le matricule et les informations obligatoires sont enregistrés.
2. Un matricule déjà utilisé est refusé ; un nom semblable produit seulement une alerte.
3. Si une donnée obligatoire manque, aucun dossier incomplet n’est créé.
4. Le dossier est retrouvable après rechargement sans exiger les informations facultatives.

US-011 · Inscrire un élève

Secrétariat · Responsable : Franck · PARCOURS

Objectif. Lier un élève à une classe pour une année précise.

Comment vérifier que cela fonctionne ?

1. L’inscription valide apparaît sur le dossier.
2. Deux demandes, même en même temps, ne créent pas deux inscriptions pour la même année.
3. Une mauvaise année, une classe déjà évaluée ou un rôle interdit bloquent l’opération.
4. On voit l’année, la classe et la trace de l’inscription.

US-012 · Voir l’historique d’un élève

Secrétariat · Responsable : Franck · PARCOURS

Objectif. Retrouver ses classes précédentes.

Comment vérifier que cela fonctionne ?

1. Les inscriptions apparaissent dans l’ordre des années.
2. Une nouvelle inscription ne supprime pas les anciennes.
3. Un élève nouvellement créé a un historique vide lisible.
4. Voir l’historique ne permet pas de changer d’anciennes notes.

US-013 · Créer un dossier enseignant

Administrateur · Responsable : Kevin · PARCOURS

Objectif. Associer un enseignant à un compte connecté.

Comment vérifier que cela fonctionne ?

1. Le compte doit être actif et porter le rôle enseignant.
2. Un compte non enseignant ou désactivé est refusé.
3. Un même compte ne reçoit pas deux dossiers enseignant.
4. Le dossier devient disponible pour les affectations sans révéler de mot de passe.

US-014 · Affecter un enseignant

Administrateur · Responsable : Kevin · PARCOURS

Objectif. Confier une matière dans une classe précise.

Comment vérifier que cela fonctionne ?

1. L’enseignant et la matière du programme existent.
2. La même matière de classe ne reçoit pas deux titulaires dans la démo.
3. Classe et année doivent correspondre.
4. Une affectation ayant déjà des évaluations ne se modifie pas librement.

US-015 · Consulter mes enseignements

Enseignant · Responsable : Kevin · PARCOURS

Objectif. Voir les classes et matières qui me sont attribuées.

Comment vérifier que cela fonctionne ?

1. La liste est filtrée selon le compte connecté.
2. Sans affectation, l’enseignant voit un message clair.
3. Changer l’identifiant dans une URL ne donne pas accès au cours d’un autre.
4. Chaque cours ouvre uniquement les évaluations concernées.

US-016 · Créer une évaluation

Enseignant · Responsable : Joel · PARCOURS

Objectif. Préparer une activité notée pour ma matière.

Comment vérifier que cela fonctionne ?

1. Titre, date, barème positif et période correcte créent un brouillon.
2. Une matière étrangère ou une mauvaise période est refusée.
3. Une classe-période fermée interdit une nouvelle évaluation, même en cas d'accès simultané.
4. La grille contient les élèves inscrits et le barème se fige dès la première note.

US-017 · Enregistrer les notes en brouillon

Enseignant · Responsable : Joel · PARCOURS

Objectif. Conserver les notes saisies avant envoi.

Comment vérifier que cela fonctionne ?

1. Un ensemble de notes correct est enregistré sans perte partielle.
2. Une note hors barème, un doublon ou un élève d'une autre classe bloque tout le lot.
3. Une note vide reste vide : elle n'est pas transformée en zéro.
4. Une grille verrouillée, trop ancienne ou appartenant à un autre enseignant est refusée.

US-018 · Soumettre les notes

Enseignant · Responsable : Joel · PARCOURS

Objectif. Envoyer une grille complète à la direction.

Comment vérifier que cela fonctionne ?

1. Tous les inscrits ont une note valide et une copie de la tentative est conservée.
2. La liste des notes manquantes apparaît si la grille est incomplète.
3. Après envoi, les notes ne se modifient plus librement.
4. Renvoyer exactement la même demande ne crée pas un deuxième envoi.

US-019 · Corriger après un rejet

Enseignant · Responsable : Joel · PARCOURS

Objectif. Corriger les notes et les soumettre à nouveau.

Comment vérifier que cela fonctionne ?

1. Le motif est visible et la grille rejetée peut être corrigée.
2. La nouvelle version conserve l'ancienne tentative.
3. Un nouvel envoi possède un numéro de tentative supérieur.
4. Une grille validée ne peut pas suivre ce parcours de correction.

US-020 · Voir les soumissions

Direction · Responsable : Melvin · PARCOURS

Objectif. Consulter les grilles en attente de décision.

Comment vérifier que cela fonctionne ?

1. La liste donne classe, matière, période, enseignant et tentative.
2. Le détail montre exactement les notes envoyées, sans modification.
3. Les décisions terminées sortent de la file mais restent dans l'historique.
4. Une personne non autorisée ne peut pas consulter les soumissions.

US-021 · Valider une soumission

Direction · Responsable : Melvin · PARCOURS

Objectif. Accepter une grille de notes.

Comment vérifier que cela fonctionne ?

1. Une tentative en attente devient acceptée et ses notes sont validées ensemble.
2. Personne ne valide ses propres notes, même avec deux rôles.
3. Deux validations concurrentes ne peuvent pas gagner toutes les deux.
4. La date, l'auteur et la trace de décision sont conservés.

US-022 · Rejeter une soumission

Direction · Responsable : Melvin · PARCOURS

Objectif. Renvoyer une grille avec une explication.

Comment vérifier que cela fonctionne ?

1. Un rejet transforme la tentative et l'évaluation en état rejeté.
2. Un rejet sans motif est impossible.
3. Une décision déjà prise ou une classe fermée bloque une autre décision.
4. L'enseignant peut lire le motif ; l'historique est conservé.

US-023 · Prévisualiser les résultats

Direction · Responsable : Melvin · PARCOURS

Objectif. Vérifier la moyenne avant la clôture.

Comment vérifier que cela fonctionne ?

1. L'exemple fictif prévu donne 13,50/20.
2. Le calcul indique sa formule et les notes utilisées ; le résultat reste provisoire.
3. Une matière absente ou une évaluation non validée est signalée.
4. Seules les notes approuvées entrent dans le calcul ; aucune note manquante ne devient zéro.

US-024 · Clôturer une classe-période

Direction · Responsable : Melvin · PARCOURS

Objectif. Figer les résultats d'une classe pour une période.

Comment vérifier que cela fonctionne ?

1. Toutes les matières nécessaires sont traitées avant clôture.
2. Si une opération échoue, aucun élève n'est partiellement clôturé.
3. Une saisie simultanée ne modifie pas les données déjà figées.
4. Une répétition de la même demande retrouve la même clôture ; pas de réouverture V1.

US-025 · Produire un bulletin

Direction · Responsable : Jonathan · PARCOURS

Objectif. Créer le document depuis un résultat figé.

Comment vérifier que cela fonctionne ?

1. Un résultat clôturé produit un bulletin marqué démonstration.
2. Sans résultat figé, aucun bulletin ne peut être créé.
3. Produire deux fois le même bulletin et modèle ne crée pas deux documents logiques.
4. L'identité et les résultats viennent de la copie figée, pas des données actuelles.

US-026 · Télécharger un bulletin

Secrétariat ou direction · Responsable : Jonathan · PARCOURS

Objectif. Retrouver et récupérer un PDF privé.

Comment vérifier que cela fonctionne ?

1. La personne autorisée télécharge le PDF.
2. Un enseignant ou une personne non connectée est refusé.
3. Changer ensuite un nom dans le dossier n'altère pas l'ancien bulletin.
4. Une panne de génération PDF se signale sans détruire le bulletin logique.

US-027 · Consulter l'historique des actions

Administrateur ou direction · Responsable : Sidonie · PARCOURS

Objectif. Savoir qui a effectué une action importante.

Comment vérifier que cela fonctionne ?

1. Le journal affiche qui, quoi, quand et la cible.
2. Les filtres permettent une recherche par période ou type d'action.
3. Aucune API ne permet de modifier ou effacer les traces ; un enseignant est refusé.
4. Aucun mot de passe, cookie ou copie complète des notes n'apparaît dans les traces.

7. Les règles qui évitent les erreurs

Les 25 règles R01-R25 du dossier original, rassemblées par sujet.

Comptes et droits

R01 · Un compte désactivé perd son accès, y compris ses sessions ouvertes.

R02 · Aucun droit n'est accordé par défaut ; l'identité vient de la session.

R03 · Personne ne valide ses propres notes, même avec plusieurs rôles.

Années, élèves et enseignants

R04 · Les périodes ont des dates valides, dans l'année et sans chevauchement pour la démo.

R05 · Chaque classe appartient à une année précise.

R06 · Au maximum une inscription par élève et par année.

R07 · Le matricule doit être unique ; deux élèves peuvent avoir le même nom.

R08 · Une classe dont les évaluations ont commencé ne change plus librement son programme ni sa liste d'inscrits.

R09 · Un enseignant possède un compte actif avec le rôle ENS.

R10 · Un seul titulaire par matière de classe pour la démo, affectation figée une fois utilisée.

Évaluations et notes

R11 · Classe, période et affectation doivent appartenir à la même année.

R12 · La note est entre zéro et le barème ; une case vide n'est pas un zéro.

R13 · La date d'évaluation est dans la période ; le barème se fige dès qu'une note existe.

R14 · Une grille doit contenir une note pour chaque inscrit avant de pouvoir être soumise.

R15 · Seul l'enseignant affecté peut modifier son brouillon ou sa grille rejetée.

R16 · Les notes soumises ou validées ne sont plus librement modifiables.

R17 · Les opérations sensibles contrôlent la version pour éviter les écrasements concurrents.

Validation et résultats

R18 · La direction accepte ou rejette ; un rejet comporte un motif et conserve l'historique.

R19 · Une moyenne provisoire utilise seulement les notes validées.

R20 · Clôture impossible tant qu'une matière obligatoire ou évaluation reste incomplète.

R21 · La clôture réalise tous les enregistrements ensemble, ou aucun.

R22 · La démo ne rouvre pas une classe-période clôturée.

Bulletins et historique

R23 · Le bulletin lit uniquement le résultat figé.

R24 · La même demande de production et le même modèle retrouvent le même document logique.

R25 · Une action importante et sa trace d'audit sont enregistrées dans la même opération.

Décision encore ouverte : évaluation abandonnée

Le dossier initial exige que chaque évaluation créée soit traitée avant la clôture. Un brouillon jamais terminé peut donc empêcher toute clôture. Nous proposons de permettre DRAFT → CANCELLED uniquement pour un brouillon non soumis, avec trace et exclusion des résultats. L'équipe doit valider précisément cette règle avant de l'implémenter.

Les états d'une évaluation

Nom technique	Affichage conseillé	Action possible
DRAFT	Brouillon	Saisir et sauvegarder ; soumettre quand la grille est complète.
SUBMITTED	Soumise	Consulter ; attendre la direction.
REJECTED	À corriger	Corriger les notes puis soumettre une nouvelle tentative.
VALIDATED	Validée	Consulter ; utiliser les notes au calcul.
CANCELLED (proposé)	Annulée	Cas encore à approuver par l'équipe ; aucune note prise en compte.

8. Comprendre le calcul de démonstration

Une formule d'exemple, pas une formule officielle scolaire.

Pour comparer des évaluations avec des barèmes différents, nous ramenons chaque note sur 20. Ensuite nous calculons la moyenne des évaluations validées d'une matière, puis la moyenne des matières prévues.

Exemple concret

Mathématiques. 15/20 et 30/40. La deuxième note équivaut à 15/20. Moyenne en mathématiques : 15/20.

Français. 12/20. Moyenne en français : 12/20.

Moyenne générale. (15 + 12) ÷ 2 = 13,50/20.

Précautions

- Aucun coefficient, rang, seuil de réussite ou passage en classe supérieure n'est inventé.
- Une matière obligatoire sans évaluation validée empêche une moyenne complète et la clôture.
- La précision du calcul est conservée avec des décimales exactes ; l'arrondi à deux chiffres n'arrive qu'à l'affichage final (HALF_UP).
- La moyenne est conservée dans ResultSnapshot ; le module bulletin ne la recalcule jamais.

> *À confirmer avec l'école : coefficients, types d'évaluation, absences, dispensés, arrondis, réussite, modèle réel du bulletin et autorité de validation.*

9. Comment les modules communiquent

Un contrat indique quelles informations un module promet aux autres.

Imaginez que Joel développe les notes et Melvin les résultats. Melvin ne doit pas deviner les champs ni modifier les tables de Joel. Ils conviennent d'un contrat simple, lisible et testé.

C01 — AuthContext

Identifiant du compte, rôles, compte actif et numéro de requête. Une vraie session validée crée ce contexte.

Sidonie → tous

C02 — SchoolContext

Année, classe, période et liste des matières attendues. Garantit leur cohérence.

Jonathan : contexte scolaire pour les autres modules

C03 — AssignmentView

Quel enseignant a droit à quelle matière et quelle classe, pour une année.

Kevin : affectations pour Joel

C04 — EnrollmentRoster

Liste stable des inscrits d'une classe, avec identifiants et matricules.

Franck : inscrits pour Joel et Melvin

C05 — SubmissionPort

Lire les tentatives, les décisions, leurs versions, et demander acceptation ou rejet.

Joel : soumissions pour Melvin

C06 — ClassPeriodGate

Vérifier et verrouiller la classe-période avant une écriture ; CLOSED interdit les changements.

Melvin : contrôle des inscriptions et des notes

C07 — ValidatedGradeSet

Liste du programme, inscrits, évaluations validées et notes nécessaires au calcul.

Joel et Jonathan : notes validées pour Melvin

C08 — FrozenResult

Résultat figé, identité figée, détail des matières, version de formule, sources et date.

Melvin : résultats figés pour Jonathan

C09 — AuditWriter

Enregistrer qui a fait quoi, sur quel objet, pendant la même transaction.

Sidonie → tous

Deux règles essentielles

- Chaque module reste responsable de l'écriture de ses propres tables. Aucun autre module ne modifie directement ses données.
- Quand une opération doit écrire plusieurs choses ensemble, les services partagent une même transaction PostgreSQL. Une erreur annule l'ensemble.

10. L'API, en termes simples

L'API est le point d'échange entre les écrans et le backend.

Un écran envoie une demande : « inscrire cet élève ». Le backend vérifie le compte, la classe, l'année et les doublons, puis répond soit avec l'inscription créée, soit avec une erreur claire.

Les grandes routes proposées

Accès

/auth/csrf, /auth/login, /auth/me, /auth/logout

Responsable : Sidonie

Comptes

/users, /roles

Responsable : Sidonie

Organisation scolaire

/school-years, /periods, /classes, /subjects, /classes/{id}/subjects

Responsable : Jonathan

Élèves

/students, /enrollments

Responsable : Franck

Enseignants

/teachers, /assignments, /me/assignments

Responsable : Kevin

Notes

/evaluations, /evaluations/{id}/grades, /evaluations/{id}/submissions

Responsable : Joel

Décisions et résultats

/submissions/{id}/decision, /results, /class-periods/close

Responsable : Melvin

Bulletins

/report-cards, /report-cards/{id}/pdf

Responsable : Jonathan

Audit

/audit

Responsable : Sidonie

Les principales réponses du backend

Code	Sens pour l'utilisateur
200 / 201	Opération réussie / nouvelle donnée créée.
204	Opération terminée sans document à afficher.
401	Vous devez vous connecter.
403	Vous n'avez pas les droits pour cette action.
409	Doublon, modification concurrente ou état incompatible.
422	Donnée saisie invalide ou règle métier non respectée.
500 / 503	Erreur technique ; ne pas afficher les secrets internes.

À conserver dans le contrat technique

L'API proposée utilise /api/v1, des identifiants UUID, les dates au format cohérent, et les notes décimales transmises comme du texte exact. Les commandes sensibles portent expectedVersion et Idempotency-Key : l'une évite d'écraser une modification récente, l'autre évite de reproduire deux fois la même action.

> *Ces routes et règles viennent du dossier source. Les schémas OpenAPI et leurs types doivent encore être créés et testés ; le dossier n'est pas un serveur en fonctionnement.*

11. Les règles de la base PostgreSQL

Protéger les données même si deux personnes cliquent en même temps.

Trois mots utiles

Clé étrangère (FK). Elle relie une donnée à une autre qui doit exister. Exemple : une inscription pointe vers un élève.

Unicité (UNIQUE). Elle empêche un doublon. Exemple : une seule inscription pour le même élève dans la même année.

Index (INDEX). Il aide à retrouver rapidement des lignes. Une contrainte UNIQUE crée déjà un index dans PostgreSQL : inutile d'en créer un identique.

Contraintes que les développeurs ne doivent pas oublier

school_year · Un code d'année ne peut apparaître qu'une fois.

period · Code et ordre uniques dans une année. Dates compatibles : contrôle protégé contre la concurrence.

school_class · Code unique à l'intérieur de l'année.

student · Matricule unique, mais nom non unique.

class_subject · Une matière ne figure qu'une fois dans le programme d'une classe.

enrollment · Un élève ne peut avoir qu'une inscription par année ; classe et année doivent correspondre.

teacher_assignment · Un seul titulaire pour une matière de classe dans la démo.

grade · Une seule note par couple évaluation + inscription.

submission · Numéro de tentative unique par évaluation ; une seule tentative en attente.

class_period · Un seul état de clôture par classe + période.

result_snapshot · Un seul résultat figé par inscription + classe-période.

report_card · Un document logique par résultat figé + version de modèle.

Index de recherche les plus utiles

Dans le dossier initial : Enrollment.classId, TeacherAssignment.teacherId, Evaluation(assignmentId,periodId), Submission(status,submittedAt), ResultSnapshot.enrollmentId et l'historique Audit par cible/date ou acteur/date.

Ce que la base ne peut pas vérifier toute seule

Une note doit viser un inscrit de la classe réellement évaluée. Une période doit appartenir à la même année que la classe. Un résultat figé doit concerner un élève inscrit dans cette classe. Ces vérifications relient plusieurs tables et demandent des contrôles du backend sous transaction, parfois des clés étrangères composées.

> *Aucun enregistrement déjà utilisé ne doit être supprimé en cascade au risque de perdre des inscriptions, notes, bulletins ou traces.*

12. Les permissions et la sécurité

Cacher un bouton n'empêche pas une requête malveillante.

À la connexion

Une session est gardée côté serveur ; le navigateur reçoit un cookie protégé (HttpOnly, et Secure en HTTPS).

Pour les actions importantes

Vérifier le rôle ET la ressource précise : un enseignant ne peut pas voir les notes d'une classe confiée à un autre.

Pour les formulaires

Prévoir une protection CSRF et contrôler l'origine des requêtes. Cela évite des actions lancées depuis un autre site à l'insu du compte connecté.

Pour les mots de passe

Ne conserver que des empreintes solides (Argon2id), jamais les mots de passe en clair ni dans les journaux.

Pour les documents

Le PDF d'un bulletin est privé ; une URL copiée ne doit pas suffire à l'obtenir sans droits.

Pour les essais

Utiliser uniquement des comptes et élèves fictifs ; ne pas placer les secrets de démonstration dans Git.

Exemple de contrôle

Si un enseignant change un identifiant dans l'adresse d'une page pour ouvrir l'évaluation d'un collègue, le serveur doit le refuser, même si l'interface avait masqué le bouton.

13. Comment vérifier que Leyisa fonctionne

Tester les situations normales, puis celles qui doivent être refusées.

Tests simples par fonction

Inscription. Deux tentatives d'inscription du même élève dans une même année ne créent jamais deux lignes.

Affectation. Un enseignant non affecté ne peut pas écrire une note.

Barème. Une note de 25 sur 20 est refusée ; zéro reste une vraie note ; une case vide reste manquante.

Soumissions. Un rejet conserve la première tentative même après correction.

Validation. Deux demandes de décision simultanées ne créent pas deux décisions ; l'auteur ne s'auto-valide pas.

Clôture. Si une note manque ou si la sauvegarde échoue au milieu, la classe reste ouverte et aucune copie partielle n'est créée.

Bulletin. Un ancien bulletin ne change pas quand l'élève change plus tard de dossier.

Session. Après déconnexion, l'ancienne session est refusée.

Redémarrage. Après l'arrêt et le redémarrage du serveur, les inscriptions, notes et bulletins sont encore présents.

Le parcours final commun

Administrateur → Secrétariat → Enseignant → Direction → Enseignant (correction) → Direction → Secrétariat. L'équipe doit pouvoir montrer tout cela dans la même application, avec de vrais comptes distincts.

> *Ces vérifications sont un plan de tests, pas des tests exécutés. La base PostgreSQL de test est nécessaire pour vérifier les vraies contraintes et transactions.*

14. Les six responsabilités de VISIO TECH

Chacun livre sa partie de bout en bout : écran, API, données, permissions et tests.

Sidonie

Responsabilité. Connexion, comptes, rôles et audit.

À montrer. Livrer une session réelle, les autorisations communes et les traces des actions importantes.

Relecture principale : Kevin

Franck

Responsabilité. Dossiers élèves, inscriptions et historique annuel.

À montrer. Éviter les doublons et fournir les inscrits utilisés pour les notes.
Conserver les inscriptions de chaque année scolaire.

Relecture principale : Jonathan

Jonathan

Responsabilité. Années, périodes, classes, matières et bulletins PDF.

À montrer. Créer la configuration et produire le PDF depuis les résultats figés.

Relecture principale : Franck ; Melvin vérifie les calculs

Kevin

Responsabilité. Enseignants, affectations et intégration technique.

À montrer. Démarrer l'environnement, aider à partager les contrats et maintenir une version intégrée utilisable.

Relecture principale : Sidonie

Melvin

Responsabilité. Validation, calcul, clôture et résultats ; lead technique.

À montrer. Valider, calculer et figer les résultats ; coordonner l'architecture.

Relecture principale : Joel

Joel

Responsabilité. Évaluations, notes, soumissions et corrections.

À montrer. Conserver les brouillons et tentatives ; protéger les notes.
Participer aux contrôles de cybersécurité avec Sidonie.

Relecture principale : Melvin

Ce qui doit être partagé dès le premier jour

Sidonie fournit C01 ; Jonathan la scolarité C02 ; Kevin les affectations C03 ;
Franck donne les inscrits C04 ; Joel, les soumissions C05 et les notes validées C07 ;
Melvin fournit la clôture C06 et les résultats figés C08 ; Sidonie l'audit C09.

15. Comment avancer en cinq jours

Ce planning remplace la proposition initiale de dix jours pour la démonstration ; ce n'est pas une garantie de livraison.

J1 - Faire démarrer ensemble

Dépôt, environnement, base PostgreSQL, sessions, données de test et contrats minimums.

Preuve visible. Une personne se connecte réellement et se déconnecte ; l'ancien accès est refusé.

J2 - Préparer la scolarité

Créer année, période, classe, matières, élèves, inscriptions et premiers enseignants.

Preuve visible. Une inscription se retrouve après rechargement et redémarrage.

J3 - Relier l'enseignement

Affectations, création des évaluations et premières grilles de notes.

Preuve visible. Le bon enseignant voit les inscrits de sa classe et pas ceux du collègue.

J4 - Soumettre et décider

Brouillons, soumissions, rejet motivé, correction et acceptation.

Preuve visible. Une première tentative rejetée reste conservée et la seconde peut être acceptée.

J5 matin - Atteindre le premier bulletin

Prévisualisation, formule DEMO_V1, clôture atomique, snapshot et PDF.

Preuve visible. Un bulletin de démonstration est produit à partir d'un résultat figé.

J5 après-midi - Tester et présenter

Droits, doublons, conflits, redémarrage, réparation des bugs et répétition.

Preuve visible. Parcours complet réalisé avec les comptes des quatre rôles.

Méthode quotidienne

Réunion rapide de 15 minutes, une seule tâche active par personne, branches Git courtes, revue par un autre membre, intégration sur main chaque jour. Une fonctionnalité « terminée » fonctionne avec l'API réelle et garde ses données en base.

> *La charge initiale du dossier était estimée pour dix jours. Avec cinq jours, il faut confirmer les disponibilités et réduire explicitement le périmètre si le premier bulletin intégré n'est pas réaliste à J5.*

16. Les décisions à prendre avant de coder

Une réunion courte suffit pour trancher ce qui bloque réellement.

1. Adopter la démonstration réduite et décider quelles interfaces seront reportées.

2. Confirmer les six responsables, leurs disponibilités et le dépôt Git commun.

3. Confirmer la stack (ou documenter clairement un changement), ses versions et le mode de démarrage.

4. Choisir la procédure de création du premier administrateur et la révocation des comptes.

5. Valider ou refuser la transition DRAFT → CANCELLED, avec traitement dans les calculs.

6. Adopter le calcul DEMO_V1 uniquement pour des données fictives et des bulletins non officiels.

7. Publier les contrats minimums C01 à C09, les données de démonstration et les règles de permissions.

8. Confirmer qui parle à l'école pour les vrais calculs, absences, valideurs et mentions du bulletin.

Ce que signifie « prêt à développer »

Nous pouvons commencer les premières fonctionnalités si les responsabilités et les échanges sont clairs. Cela ne veut pas dire que la future application est déjà validée, sécurisée ou prête à recevoir de vraies données d'élèves.

Petit lexique, pour ne pas perdre le fil

Les termes utiles sont expliqués à partir d'exemples Leyisa.

Backend. Partie qui vérifie les règles et enregistre les données.

Frontend. Écrans utilisés par le secrétariat, les enseignants et la direction.

API. Chemin par lequel les écrans demandent une action au backend.

Module. Groupe de fonctions qui traitent un sujet, par exemple les notes.

Base de données. Lieu où les informations sont conservées, y compris après redémarrage.

User Story (US). Besoin concret d'un utilisateur, avec des conditions pour dire qu'il est réussi.

MVP / Démo. Petite version permettant de montrer un parcours réel, sans prétendre tout gérer.

RBAC. Règles indiquant ce que chaque rôle peut voir ou faire.

Transaction. Ensemble d'enregistrements effectués entièrement ou annulés en bloc.

Concurrence. Deux personnes agissent au même moment : le logiciel doit rester cohérent.

Index. Organisation interne de la base pour retrouver plus vite certaines lignes.

Snapshot. Copie figée des résultats au moment de la clôture.

DRAFT. Évaluation en brouillon, encore modifiable.

CANCELLED. Évaluation annulée ; état encore proposé et non adopté pour Leyisa.

CSRF. Protection des formulaires contre certaines actions lancées sans l'accord de l'utilisateur.

Idempotence. Répéter la même demande ne produit pas une deuxième opération.

OpenAPI. Document partagé qui décrit précisément les routes et les données de l'API.

E2E. Test complet allant de l'écran jusqu'à la base et au résultat visible.

Références et limites

Ce document doit rester cohérent avec les décisions de l'équipe.

Document de base : « Leyisa School V1 — Dossier de préparation au développement », 77 pages, daté du 7 octobre 2026, fourni par l'équipe. Cette réécriture en conserve les identifiants US-001 à US-027, R01 à R25 et C01 à C09.

Support visuel complémentaire : « Leyisa School V1 — UML corrigé V4 ». Les relations et multiplicités doivent être vérifiées dans la version UML, pas déduites des exemples simplifiés de ce guide.

Ce que cette version actualise

Le projet vise maintenant une démonstration à organiser autour de cinq jours restants, au lieu du sprint théorique de dix jours du premier document.

Le premier bulletin intégré est proposé pour J5 ; la fin de J5 est réservée aux tests et corrections.

- L'annulation d'un brouillon abandonné est clairement signalée comme une proposition qui attend validation.
- Les rôles, le calcul et le modèle du bulletin restent des hypothèses de démonstration, non des règles officielles de l'école.

Sources techniques indiquées dans le dossier initial

NestJS — Authorization : https://docs.nestjs.com/security/authorization

NestJS — OpenAPI security : https://docs.nestjs.com/openapi/security

Prisma — Transactions : https://www.prisma.io/docs/orm/prisma-client/queries/transactions

OWASP — Session Management Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

> *Aucun code applicatif n'a été modifié, aucun test backend n'a été exécuté et aucune règle scolaire réelle n'a été vérifiée lors de cette réécriture documentaire.*

Annexe · Comprendre les UML (1/2)

Ces explications vont avec les dessins du document « Leyisa School V1 — UML V4 ». Elles ne remplacent pas les schémas.

Lire les chiffres : « 1 » = exactement un ; « 0..1 » = zéro ou un ;
« 0..* » = aucun, un ou plusieurs. Une flèche montre un échange ou une étape.

01 Cas d’utilisation

UML V4 · p. 2

Idée : Les personnages sont les utilisateurs ; les ovales sont les actions qu’ils peuvent demander.

Exemple : Le secrétariat inscrit un élève ; la direction valide les notes. Chaque accès reste contrôlé.

02 Classes · Scolarité

UML V4 · p. 3

Idée : Le dossier Student reste le même ; Enrollment le rattache à une classe pour une année. ClassSubject définit les matières de cette classe.

Exemple : Amina est inscrite en 6e A cette année ; les maths de 6e A sont confiées à un enseignant.

03 Classes · Notes et envois

UML V4 · p. 4

Idée : Une évaluation concerne une matière affectée et une période. Chaque Grade concerne un élève inscrit. Submission garde les tentatives.

Exemple : L’interro de maths est notée sur 20 ; la grille refusée reste conservée même après correction.

04 Classes · Clôture et bulletin

UML V4 · p. 5

Idée : ClassPeriod représente une classe pendant une période. ResultSnapshot fige chaque résultat ; ReportCard affiche ce résultat sans le recalculer.

Exemple : Après la clôture de 6e A, le bulletin d’Amina retrouve sa moyenne même si son dossier évolue.

Annexe · Comprendre les UML (2/2)

Suivre les flèches aide à comprendre le parcours ; les cases, états et contraintes empêchent les erreurs.

05 Activité · Saisie et décision

UML V4 · p. 6

Idée : Les colonnes montrent qui fait quoi : enseignant, application, direction. Un losange correspond à une décision.

Exemple : Si une note manque, la grille retourne au brouillon ; si la direction rejette, elle indique un motif.

06 États d'une évaluation

UML V4 · p. 7

Idée : DRAFT est modifiable ; SUBMITTED attend une décision ; REJECTED permet de corriger ; VALIDATED fige la grille.

Exemple : Une grille rejetée peut être corrigée et renvoyée. CANCELLED reste seulement une proposition.

07 Séquence · Clôture atomique

UML V4 · p. 8

Idée : Le dessin suit les échanges dans le temps. En transaction, soit tous les résultats sont figés avec la clôture, soit rien ne change.

Exemple : Si la 6e A a une note manquante, la clôture échoue : aucun résultat partiel n'est enregistré.

08 Base · Contraintes et index

UML V4 · p. 9

Idée : UNIQUE évite certains doublons ; les index accélèrent les recherches. Les liens entre objets restent contrôlés par le backend.

Exemple : Deux inscriptions du même élève dans la même année sont refusées, même en cas de double clic.

À retenir

Les diagrammes expliquent le fonctionnement prévu, pas du code déjà testé. Les calculs DEMO_V1, les autorisations et l'état CANCELLED attendent les validations indiquées dans le guide.